Treasure Maze (Junior Edition): Teacher Guide

At a Glance

Age	8 to 9, using the **Junior** edition of PyGameMaker
Time	90 minutes. This is the second maze lesson: do "My First Maze" first if you can
Students make	A treasure maze: draw the hero, create the objects, build a maze with doors, gems, a key, a heart and bombs, program it, export it as one .html file
Needs	PyGameMaker in Junior mode, the starter project, a web browser. Internet is **not** needed to play this game
Student sheet	Junior Treasure Maze: Student Handout (one for each student)

Everything in the lesson uses only blocks that exist in the Junior toolbox. The finished game was run in the real game engine, exported, and played in a browser to check every mechanic.

> This lesson is ambitious for 8 to 9 year olds in 90 minutes. If your class is slow, stop after 5b (gems) and finish the rest next time. The priorities are listed in the lesson plan.

Before the Lesson (15 minutes, once)

1. **Switch on Junior mode** on each computer: Tools > Preferences > General > IDE Edition > Junior, then restart the IDE.
2. **Copy the starter project** (Junior-Treasure-Starter.zip, from the wiki page or wiki/downloads/) and unzip it so each student has a folder called **Treasure Maze**.
3. **Try the lesson yourself once.** The finished version is in Junior-Treasure-Finished.zip to compare with.
4. **Use an up-to-date PyGameMaker for the export.** Older builds of the HTML5 export did not support "Destroy all instances of", so the key would vanish without opening the doors in the browser. This was fixed together with this lesson.
5. **Decide how the game will get to a phone** (see "Playing on a Phone").
6. Print the student handout, or put it on a second screen.

What the Starter Contains

- **Seven ready-made pictures**: spr_wall, spr_door, spr_gem, spr_key, spr_heart, spr_bomb, spr_goal.
- **obj_wall**, with spr_wall, **not yet Solid**. It has to exist because the room's border is built from it. Students tick Solid in Part 3.
- **room0**: 640 by 416 pixels, on a 32-pixel grid, with the border built and nothing inside.

Students draw only the hero. Everything else is theirs to create, build and program.

Lesson Plan (90 minutes)

Time	Part	What happens	If you are late
0-5	Warm-up	Play the finished game. Ask: what pieces does it need?	Skip the discussion
5-10	1. Open	Open the starter, look at the resources, F5	
10-20	2. Hero	Create Sprite, draw the hero	Give a ready hero from the Finished project
20-32	3. Objects	Tick Solid on the wall, create 7 objects	Work in pairs: one reads the table, one clicks
32-47	4. Maze	Paint walls, place door, key, gems, bombs, heart, goal, hero	Give the Finished room
47-55	5a. Hero	Seven rows of blocks	**Priority 1**
55-59	5b. Gems	Score, then disappear	**Priority 1**
59-63	5c. Key	Destroy all the doors	Priority 2
63-66	5d. Heart	One more life	Priority 3: skip first
66-72	5e. Bombs	Lose a life, back to the start	Priority 2
72-78	5f. Goal	The "no gems left" question	**Priority 1**
78-80	5g. Game over	Other event: No More Lives	Priority 3: skip first
80-87	6. Export	Export as HTML5, open it in the browser	Do it as a class demo
87-90	Wrap-up	Swap and play; the "Think About It" questions	

If time is short, go from 5b straight to 5f: the hero, the gems and the goal already make a complete game. Then add the key, the bombs, the heart and the game over in that order.

Walkthrough and Common Problems

Part 2, the hero. Create Sprite opens a blank 32 by 32 picture. The collision box follows the drawn (non-transparent) pixels, so a hero that fills the square is the biggest and a hero with empty edges slips through paths more easily. Both work in two-square paths.

Part 3, the objects. Eight objects in all. Only the wall and the door are Solid.

> In this game the **collision rows** stop the hero, not the Solid setting: a Solid wall alone does not stop it (checked: the hero walks through). We still ask for Solid because it is the correct setting for walls and doors, and some other blocks, such as Bounce, depend on it.

Part 4, the maze.

- **Two squares wide.** The hero stops wherever a key is released, not on the grid. In a one-square path it can end up a few pixels off and jam. The reference maze has paths two or three squares wide; a test plays it with the hero placed up to 16 pixels off the grid.
- **A door** is a column of door objects across a path, as wide as the path. Students often place only one and leave a gap: the hero walks around it.
- **The key must come first.** It must be reachable without passing a door. Check each maze before swapping.
- **Bombs** go along the edge of a path, never across it. The reference maze puts each bomb on the edge row of a three-square path.

Part 5, programming.

- 5a, the hero: as in "My First Maze". Without the two **collision** rows the hero walks through walls and doors.
- 5b, gems: the gem does the work: *when colliding with obj_hero: Add to score 10, Destroy this instance*.
- 5c, the key: **Destroy all instances of** obj_door removes every door in the room. Then **Destroy this instance** removes the key.
- 5e, bombs: the **hero** has the event, so **Destroy other instance** means the bomb. The four blocks run in order: lose a life, bomb disappears, stop, go home. Without **Stop movement** the hero keeps moving if a key is held.
- 5f, the goal: the **If** block asks how many gems exist (*count of object obj_gem == 0*). Blocks placed in its **then** slot run when the answer is yes, blocks in its **else** slot when it is no. This is the one new idea of the lesson: take your time with it. The block starts with the object name obj_coin, so students must type obj_gem. Its words (If, count of, then, else) stay in English even when the IDE is in French.
- 5g, game over: the **Other event** block has a drop-down; choose **No More Lives**. **Restart Game** starts everything again: score 0, 3 lives, every gem, door, key, heart and bomb back (checked in the game engine and in a browser).

Exporting

File > Export as HTML5 asks for a folder and offers to open the game in the browser. The result is **one file**, named after the project. This game has no Python code, so it needs no internet to play. The bar above the game shows **Score** and **Lives**, which this game really uses.

Playing on a Phone

What is known:

- The exported page detects a touchscreen and then shows four arrow buttons under the game. Holding a button moves the hero. This was checked in a phone-sized browser on a computer, **not on a real phone**.
- The game page is built for narrow screens and shrinks to fit.
- Every mechanic of this game (gems, key and doors, heart, bombs, goal, game over) was checked in a headless browser by moving the hero onto each item.

What to plan for:

- **Getting the file onto the phone.** Phones often cannot open an .html file from their storage (iPhones in particular). The dependable way is to put the file on a **web address** (the school's web space or a free site host), then open that address on the phone. This was not tested here.
- **Speed on a fast screen.** The exported game takes one step per screen refresh. On a 60 Hz screen this is the intended speed. On a phone with a 90 or 120 Hz screen the hero will move faster, up to twice as fast. This is a known limit of the HTML5 export that has not been fixed. This maze is still playable, but bombs are harder to dodge at double speed.

Differentiation

- **Needs support:** start from the Finished room and only change the walls, or give a printed maze to copy. Do priorities 1 and 2 only.
- **Going further:** a second key and a second set of doors, a gem hidden behind a door, or more points for some gems. All use blocks in the Junior toolbox.

Assessment

Ask each student to show you: the hero they drew, a maze that can be finished, at least three working mechanics (for example gems, a door with a key, and a bomb), and the exported .html file working in a browser. The three "Think About It" questions work as an exit ticket.

Troubleshooting

Symptom	Cause and fix
Hero goes through walls or doors	Missing collision row on the hero (**When colliding with** obj_wall or obj_door, **Stop movement**)
Hero stuck in a path	Path is one square wide. Widen it
Key does nothing	**Destroy all instances of** obj_door missing or set to the wrong object
The door can never be opened	The key is behind the door. Move it
Goal always says "Collect every gem first!"	A gem is somewhere unreachable, or the number in the **If** block is not 0
Goal wins at once	The **If** block is missing, or its object is not obj_gem (it starts as obj_coin)
Bomb does nothing	The event is on the hero: **When colliding with** obj_bomb
Hero cannot dodge a bomb	The bomb blocks the whole path. Move it to the edge
Doors stay after the key in the browser	An old build of the export. Update PyGameMaker and export again
Grey game window	Slow or low-memory computer. Wait, then try again
Something deleted	Tools > Restore Deleted Assets